Documento de Visão

Versão 1.0

Revisões do Documento

Revisões são melhoramentos na estrutura do documento e também no seu conteúdo. O objetivo primário desta tabela é a fácil identificação da versão do documento. Toda modificação no documento deve constar nesta tabela.

Versão	Data	Autores	Observações
1.0	19/03/2026	Maria Fernanda L. R. de Sousa, Maria Clara Moita, Luiz Guilherme e João Gabriel	Foram adicionados no do documento de visão a introdução, com finalidade, escopo, definições e referências, a contextualização do problema e os stakeholders.

Sumário

1. Introdução 4
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 5
1.4 Referências 5
1.4.1 Especificações Suplementares 5
2.1 Descrição do Problema 6
2.2 Sentença de Posição do Produto 6

3. Descrição dos *Stakeholders* e dos Usuários 7

3.1 Principais *Stakeholders* e Usuários 7
3.2 Necessidades Chave dos *Stakeholders* e dos Usuários 8

Documento de Visão

1. Introdução

1.1 Finalidade

A finalidade deste documento é apresentar a visão geral do sistema de detecção de credenciais em repositórios, definindo como os stakeholders enxergam a solução, suas funcionalidades principais e as necessidades que ela busca atender.

O sistema proposto consiste em um scanner automatizado de credenciais e dados sensíveis em repositórios de código, com execução antes do commit, com o objetivo de prevenir vazamentos acidentais de informações críticas.

A principal proposta é ir além dos scanners tradicionais, que utilizam apenas padrões simples, incorporando uma análise mais inteligente baseada em contexto. O sistema será capaz de identificar não apenas segredos explícitos, como senhas e chaves, mas também dados ofuscados ou escondidos, além de analisar o uso dessas informações dentro do código.

A ferramenta será integrada ao fluxo de desenvolvimento por meio de um *pre-commit hook* do Git, atuando de forma preventiva e educativa, alertando o desenvolvedor ou bloqueando o commit quando necessário.

1.2 Escopo

1.2.1 Objetivo Geral

Desenvolver um sistema capaz de identificar e prevenir o vazamento de credenciais e dados sensíveis em repositórios de código, por meio da análise automática de arquivos antes da realização de commits no Git.

1.2.2 Objetivos específicos:

- Expandir os formatos de arquivos a serem analisados: O sistema será capaz de analisar arquivos de código-fonte, arquivos de configuração (.env, .json, .yaml, .xml), dumps de banco de dados (.sql), arquivos de log (.log), arquivos de chave (.pem, .key, .crt) e arquivos compactados (.zip);
- Aprimorar a análise de formatos de arquivos já incluídos em scanners comuns;
- Proporcionar conhecimento com relação a correções e implicações do erro: Proporcionar ao desenvolvedor orientações sobre boas práticas de segurança e correção de vulnerabilidades identificadas;
- Identificar padrões específicos em dados sensíveis;
- Integrar o sistema ao fluxo de desenvolvimento por meio de hooks do Git, permitindo a execução automática antes dos commits;
- Detectar vazamentos em arquivos não convencionais: Ao identificar possíveis dados sensíveis, o sistema deverá gerar alertas ao desenvolvedor, podendo impedir a realização do commit;
- Integrar ao fluxo de desenvolvimento;
- Notificar e orientar o desenvolvedor: Emitir alertas claros e fornecer sugestões de correção para evitar o vazamento de dados sensíveis;
- Reduzir risco de exposição de dados;
- Propor melhorias em relação às ferramentas existentes.

1.2.3 Planejamento de recursos:

- Recursos Humanos
  - Desenvolvimento
  - Testes
  - Documentação

- Recursos tecnológicos
  - Linguagem: Python
  - Versionamento: Git

- Repositório: Github
- Ambiente de desenvolvimento: Visual Studio Code

❖ Recursos computacionais
- Computador pessoal com capacidade para execução de scripts Python e manipulação de arquivos locais.

❖ Tempo
- Planejamento de requisitos
- Desenvolvimento do produto
- Testes
- Documentação

❖ Delimitação do escopo
- O sistema não contempla análise de arquivos multimídia, como imagens e vídeos, nem técnicas avançadas como OCR, sendo essas consideradas possíveis evoluções futuras.

1.3 Definições, Acrônimos e Abreviações

- **Git**: Sistema de controle de versão distribuído utilizado para gerenciar código-fonte.
- **Commit**: Ação de salvar alterações no repositório.
- **Pre-commit hook**: Script executado automaticamente antes da confirmação de um commit.
- **API Key**: Chave utilizada para autenticação em serviços e APIs.
- **Token**: Código de autenticação usado para acesso a sistemas.
- **Base64**: Método de codificação de dados frequentemente utilizado para ofuscação.
- **Entropia**: Medida utilizada para identificar o nível de aleatoriedade de uma string, podendo indicar um possível segredo.

Mais informações poderão ser detalhadas futuramente no glossário do sistema.

1.4 Referências

1.4.1 Especificações Suplementares

- Git. *Git Documentation*. Git SCM. Disponível em: https://git-scm.com/docs. Acesso em: 20 mar. 2026.
- Git. *Customizing Git – Git Hooks*. Disponível em: https://git-scm.com/docs/githooks. Acesso em: 20 mar. 2026.
- OWASP. *OWASP Top 10 – Web Application Security Risks*. Disponível em: https://owasp.org/www-project-top-ten/. Acesso em: 20 mar. 2026.
- OWASP. *Secrets Management Cheat Sheet*. Disponível em: https://cheatsheetseries.owasp.org/. Acesso em: 20 mar. 2026.
- Lei Geral de Proteção de Dados Pessoais. Lei nº 13.709, de 14 de agosto de 2018. Disponível em: https://www.planalto.gov.br/. Acesso em: 20 mar. 2026.

- GitHub. *Secret Scanning Documentation*. Disponível em: https://docs.github.com/. Acesso em: 20 mar. 2026.

2. Contextualização

Atualmente, um dos principais problemas enfrentados no desenvolvimento de software é o vazamento acidental de credenciais e dados sensíveis em repositórios de código. Esse tipo de falha ocorre, na maioria das vezes, por descuido dos desenvolvedores ao incluir informações como senhas, tokens, chaves de API ou arquivos de configuração diretamente no código.
As ferramentas existentes para detecção de segredos apresentam limitações relevantes, principalmente relacionadas à alta taxa de falsos positivos, pois utilizam apenas padrões simples baseados em expressões regulares. Isso faz com que muitas strings sejam identificadas como ameaças, mesmo quando não representam risco real, reduzindo a confiança dos desenvolvedores nessas ferramentas.

2.1 Descrição do Problema

Problemas	• Vazamento acidental de credenciais (senhas, tokens, API keys) em repositórios de código • Alta taxa de falsos positivos em ferramentas atuais de detecção • Dificuldade em identificar segredos ofuscados (Base64, hexadecimal, etc.) • Falta de análise de arquivos não-código (ex: *.env*, dumps, planilhas) • Ausência de verificação eficiente no histórico do repositório • Baixa conscientização dos desenvolvedores sobre boas práticas de segurança
Pessoas Atingidas	• Desenvolvedores de software • Equipes de Segurança da Informação (AppSec) • Empresas e organizações que utilizam repositórios de código • Equipes de DevOps e gestores de TI
Cujo impacto é	• Exposição de dados sensíveis e credenciais • Risco de ataques cibernéticos e acessos não autorizados • Prejuízos financeiros e danos à reputação da organização • Retrabalho para remoção de segredos já versionados • Redução da confiança em ferramentas de segurança devido a falsos positivos
Uma solução bem-sucedida traria	• Prevenção de vazamentos antes do commit • Redução de falsos positivos com análise de contexto • Detecção de segredos explícitos e ofuscados • Maior segurança nos repositórios de código • Conscientização dos desenvolvedores sobre boas práticas • Integração simples ao fluxo de desenvolvimento sem impactar a produtividade

2.2 Sentença de Posição do Produto

Para	equipes de desenvolvimento de software, profissionais de segurança da informação (AppSec) e organizações que utilizam repositórios de código
O	scanner inteligente de credenciais em

	repositórios
Que	identifica e previne o vazamento de dados sensíveis, como senhas, tokens e chaves de API, por meio de análise baseada em padrões e contexto, atuando antes da realização de commits
Diferente de	ferramentas tradicionais de detecção de segredos que utilizam apenas padrões simples (regex) e atuam de forma reativa
Nosso produto	oferece uma abordagem preventiva e mais inteligente, integrando-se ao fluxo de desenvolvimento via hooks do Git, utilizando análise contextual e técnicas como entropia para detectar inclusive informações ofuscadas, além de atuar de forma educativa ao alertar o desenvolvedor e reduzir riscos de vazamentos

3. Descrição dos *Stakeholders* e dos Usuários

3.1 Principais *Stakeholders* e Usuários

Identificação	Responsabilidades	*Stakeholders*
Desenvolvedor de Software	Implementar código, realizar commits e tratar os alertas de segurança gerados pelo scanner antes de enviar o código ao repositório.	Usuário
Líder Técnico (Tech Lead)	Definir as regras de bloqueio, configurar os níveis de sensibilidade do scanner e garantir que a ferramenta seja adotada por toda a equipe.	Stakeholder
Equipe de Segurança (AppSec)	Validar os padrões de detecção (regex/entropia), auditar possíveis falsos positivos e definir os critérios de conformidade de segurança.	Stakeholder
Gestor de TI / Coordenação	Garantir que o projeto minimize riscos de vazamento de dados (LGPD) e validar o impacto da ferramenta na produtividade da equipe.	Stakeholder

3.2 Necessidades Chave dos *Stakeholders* e dos Usuários

No.	Descrição	Prioridade do Cliente (Crítico, Útil e Importante)	Observações
1	Detectar automaticamente credenciais e dados sensíveis (senhas, tokens, API Keys) no código antes do commit	Crítico	Funcionalidade principal do sistema
2	Integrar o scanner ao fluxo de desenvolvimento por meio de hooks do Git	Crítico	Deve funcionar de forma transparente para o desenvolvedor
3	Identificar segredos mesmo quando estiverem ofuscados (ex: Base64 ou strings aleatórias)	Crítico	Uso de análise por entropia e contexto
4	Alertar o desenvolvedor de forma clara e educativa sobre possíveis vazamentos	Importante	Evitar frustração e melhorar adoção
5	Permitir configuração de níveis de sensibilidade e regras de detecção	Importante	Necessidade do Tech Lead e equipe de segurança
6	Minimizar falsos positivos para não impactar a produtividade da equipe	Crítico	Essencial para aceitação da ferramenta
7	Garantir conformidade com boas práticas de segurança e proteção de dados	Importante	Alinhamento com políticas e normas
8	Gerar logs ou relatórios simples para auditoria de segurança	Útil	Apoio para equipe de segurança (AppSec)